AIRPORT FUEL MANAGEMENT

Invoice flow, frontend to backend

A concise code walkthrough of how AFM loads invoice data, calculates balances, and turns the invoice screen into a print-ready PDF.

React + FastAPI + SQLAlchemy

What happens when someone prints an invoice?

1

Open invoice

React reads the invoice ID from the route.

2

Fetch details

Frontend requests the authenticated invoice API.

3

Render document

React places returned values in the invoice layout.

4

Print / save PDF

Browser print styles format the page as A4.

Frontend responsibilities

Load and display

`InvoiceDetail.jsx` reads `id` with React Router, calls `GET /invoices/{id}`, and renders the reference, dates, airline, supplier, fuel line, and totals.

- React state holds invoice data, loading/error state, and payment form values.
- Currency and status labels use shared formatting helpers.
- Finalize, cancel, and payment actions call API endpoints and refresh the displayed data.

Print-ready styling

`style.css` defines the branded invoice and its print version.

- Metadata and airline/provider details are grouped into clear cards.
- Payment status and due date sit beside the totals summary.
- `@page` sets A4; print rules hide navigation and controls marked `no-print`.
- The Print button calls `window.print()`; the browser creates the PDF.

PDF generation detail: AFM does not generate a PDF on the server. It serves invoice data; the browser prints the React invoice view using print-specific CSS.

SERVER-SIDE DATA AND CALCULATIONS

Backend responsibilities

FastAPI returns a stable invoice response. SQLAlchemy loads the invoice record, and the response schema adds payment totals and status.

```
GET /api/v1/invoices/{invoice_id}   →   InvoiceOut
```

API endpoint

`backend/app/main.py` defines the authenticated invoice detail endpoint. It looks up the invoice by ID, returns 404 when missing, and serializes it through `InvoiceOut`.

The frontend API client sends session credentials and includes a CSRF token for write requests.

Response schema

`backend/app/schemas/current.py` defines the fields the frontend receives and computes:

- **paid_amount** = sum of recorded payments
- **balance_due** = invoice total − paid amount
- **payment_status** = derived from invoice state and balance

Keeping invoice history stable

`backend/app/services.py` creates an invoice using the applicable fuel rate, calculates the subtotal with monetary rounding, and stores the rate, quantity, currency, tax, and totals on the invoice.

The invoice screen displays these saved values; it does not recalculate totals from today's fuel rate.

Key files

`frontend/src/pages/InvoiceDetail.jsx`
Invoice view, loading, print action, and payment interactions.

`frontend/src/style.css`
Screen layout, responsive behavior, and A4 print rules.

`frontend/src/api.js`
Axios config, credentials, CSRF, and error handling.

`backend/app/main.py`
Authenticated invoice detail and action endpoints.

`backend/app/schemas/current.py`
Invoice response model and computed payment values.

`backend/app/services.py`
Rate selection, invoice snapshots, and amount calculations.

30-SECOND SUMMARY FOR THE DEMO

"The backend owns invoice records and payment calculations. The React page fetches the invoice through an authenticated API and renders a print-ready view. CSS switches that view to A4 and hides app controls when printing. The browser exports it as a PDF, while the saved invoice snapshot keeps historical amounts stable if fuel rates change."